Documento de Apoio – Minitrabalho DAA 2025/2026

(Versão 1)

Conteúdo

1 Diâmetro de um Grafo 1

2 Degree Centrality (DC) 2

3 Closeness Centrality (CC) 2

4 Eigenvector Centrality (EC) – *Power Iteration* 3

5 Betweenness Centrality (BC) – Algoritmo de Brandes 5

6 Referências 7

1 Diâmetro de um Grafo

O **diâmetro** de um grafo conexo, $G = (V, E)$ é a maior distância (em número de arestas) entre qualquer par de vértices:

$$\mathrm{diam}(G) = \max_{u,v \in V} d(u, v) \tag{1}$$

Para calculá-lo, é útil o conceito de **excentricidade** de um vértice v:

$$\varepsilon(v) = \max_{u \in V} d(v, u) \tag{2}$$

O diâmetro é então $\mathrm{diam}(G) = \max_{v \in V} \varepsilon(v)$.

Interpretação: numa rede social, o diâmetro é o "pior caso" de separação entre dois personagens. Está diretamente relacionado com a teoria dos *seis graus de separação* (Milgram, 1967)[1]: se o diâmetro for próximo de 6, a rede exibe esta propriedade de "mundo pequeno". Em redes reais, o diâmetro cresce tipicamente como $O(\log n)$ com o número de vértices, graças à presença de *hubs* (ou seja, vértices com um número elevado de ligações, portanto, com um grau muito superior à média da rede).

Notas de implementação

- O diâmetro deve ser calculado apenas sobre a **maior componente conexa**; em grafos desconexos, o diâmetro global seria infinito.
- A excentricidade de v, $\varepsilon(v)$, pode ser calculada a partir da informação das distâncias dos vértices guardada ao efetuar uma travessia em largura (BFS) a partir de v: basta tomar o valor máximo de todas as distâncias. Também poderá dar jeito guardar o conjunto de vértices do caminho que realiza a distância máxima.
- Portanto, para encontrar o diâmetro basta efetuar uma BFS por cada vértice da maior componente conexa: o diâmetro é o máximo das excentricidades encontradas.

[1] https://en.wikipedia.org/wiki/Six_degrees_of_separation

2 Degree Centrality (DC)

A **Degree Centrality** é a métrica mais direta de centralidade: normaliza o grau de cada vértice pelo máximo possível num grafo simples com n vértices.

$$DC(v) = \frac{\deg(v)}{n-1} \tag{3}$$

Com esta normalização, $DC(v) \in [0, 1]$, e $DC(v) = 1$ significa que v está ligado a todos os outros vértices.

Interpretação: numa rede social, DC identifica as pessoas com mais interações diretas e fornece uma noção intuitiva de "popularidade" ou "número de contatos".

3 Closeness Centrality (CC)

A **Closeness Centrality** mede quão próximo um vértice está de todos os outros, em termos de distância de caminho mínimo (em número de arestas).

Para grafos **conexos**, a definição clássica é a recíproca da soma das distâncias:

$$CC_{\text{clássica}}(v) = \frac{r_v}{\sum_{u \neq v} d(v, u)} \tag{4}$$

onde r_v é o número de vértices *alcançáveis* a partir de v (excluindo o próprio v) e $d(v, u)$ é a distância mínima em número de arestas

Para grafos **desconexos** (como os deste trabalho), esta fórmula não é diretamente aplicável porque vértices inacessíveis teriam distância infinita. A variante de **Wasserman e Faust** resolve isto penalizando vértices em componentes pequenas:

$$CC(v) = \frac{r_v^2}{(n-1) \cdot \sum_{u \neq v} d(v, u)} \tag{5}$$

onde r_v é o número de vértices *alcançáveis* a partir de v (excluindo o próprio v) e $d(v, u)$ é a distância mínima em número de arestas. Os **pesos do CSV não são usados**, portanto, cada aresta vale 1, independentemente do peso de co-ocorrência.

O fator r_v^2 no numerador penaliza vértices em componentes conexas pequenas: mesmo que v seja o centro da sua componente, o seu CC será baixo se a componente tiver poucos vértices. O fator $(n-1)$ normaliza em relação ao tamanho total do grafo.

Interpretação: numa rede social, uma pessoa com CC elevada consegue alcançar toda a rede em poucas ligações, sendo, portanto, uma boa "propagadora de informação". Em redes com propriedades de "Pequeno Mundo" (ou seja, caminhos médios curtos entre vértices e forte ligação dos vértices com os vértices vizinhos), a métrica CC consegue distinguir os nós centrais dos periféricos, mesmo quando têm graus semelhantes. Esta métrica difere da métrica DC, pois penaliza os vértices na periferia da rede, mesmo que tenham muitos vizinhos diretos.

Atenção: use a BFS (não Dijkstra) para calcular distâncias, uma vez que o grafo é não pesado para efeitos desta métrica.

Notas de implementação

- Se ao efetuar uma travessia em largura (BFS) a partir de um vértice v, for guardando a informação de distâncias de todos os vértice alcançáveis a partir de v numa variável, por exemplo, `dist`, temos:
  - r_v = número de entradas em `dist`.
  - $\sum_{u \neq v} d(v, u)$ = a soma de todos os valores em `dist`
- *Caso especial:* Caso o vértice esteja isolado na rede, (ou seja, não alcança nenhum outro vértice no grafo), atribua $CC(v) = 0$.
- Garanta que a soma de distâncias nunca inclui vértices que não são alcançáveis por v.

4 Eigenvector Centrality (EC) – *Power Iteration*

Uma classe mais sofisticada e importante de medidas de centralidade baseia-se na ideia de que um nó é importante se estiver ligado a outros nós importantes.

Por esta razão, este tipo de métricas não podem ser calculadas separadamente para cada vértice; em vez disso, calculamo-las para todos os nós simultaneamente, recorrendo a um sistema de equações.

Estas métricas são coletivamente denominadas "centralidade baseada em vetores próprios", pois o seu cálculo envolve vetores próprios. Estas métricas são amplamente utilizadas, por exemplo:

- Para classificar as páginas da web (o famoso algoritmo da Google, o PageRank[2]);
- Para verificar quais os agentes numa rede social que são influentes na formação da opinião de consenso a longo prazo do grupo (conhecido como "Modelo de DeGroot"[3]).

A métrica EC parte do princípio de que a centralidade de um vértice (dado por um vetor não nulo) é proporcional à soma da centralidade dos seus vizinhos, ou seja:

$$\lambda\, EC(v) = \sum_{u \in \mathrm{viz}(v)} EC(u) \tag{6}$$

Exemplo (com grafo orientado). Considere o seguinte grafo orientado com matriz de adjacência A:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \qquad \begin{array}{l} \text{nó 1 liga a 2 e 3} \\ \text{nó 2 liga a 1 e 3} \\ \text{nó 3 não liga a ninguém} \end{array}$$

A EC satisfaz o sistema de equações:

$$\lambda c_1 = c_2$$

$$\lambda c_2 = c_1$$

$$\lambda c_3 = c_1 + c_2$$

com $c_1 + c_2 + c_3 = 1$. A solução é, portanto: $\lambda = 1$, $c_1 = c_2 = \frac{1}{4}$, $c_3 = \frac{1}{2}$.

Apesar de o nó 3 não ter saídas, é o mais central, pois todos os caminhos de 1 e 2 passam por ele.

[2] https://pt.wikipedia.org/wiki/PageRank
[3] https://en.wikipedia.org/wiki/DeGroot_learning

Em notação matricial, sendo A a matriz de adjacência e $\mathbf{x}$ o vetor de scores, esta equação é $A\mathbf{x} = \lambda\mathbf{x}$: o vetor de `scores` é um **vetor próprio** de A associado ao maior valor próprio λ_1. O Teorema de Perron-Frobenius garante que, para grafos conexos, este vetor próprio existe, é único (a menos de escala) e tem todas as componentes positivas.

Porquê não resolver a equação característica?

A abordagem algébrica direta seria encontrar λ resolvendo a equação característica, um polinómio de grau n em λ cujas raízes são os valores próprios de A: $\det(A - \lambda I) = 0$. No entanto, esta abordagem é impraticável por duas razões fundamentais:

1. *Custo computacional proibitivo:* Calcular o determinante de uma matriz $n \times n$ tem complexidade $O(n^3)$, mais o custo para encontrar as raízes de um polinómio de grau n.
2. *Instabilidade numérica:* Para n grande, os coeficientes do polinómio característico têm magnitudes astronomicamente diferentes, tornando qualquer avaliação numérica extremamente sensível a erros de arredondamento.

Motivação matemática para o *Power Iteration*

A chave está em observar o que acontece quando se multiplica repetidamente um vetor inicial não negativo, $\mathbf{x}^{(0)} \in \mathbb{R}^n$, pela matriz A. Qualquer vetor inicial pode ser escrito como uma combinação linear dos vetores próprios $\mathbf{v}_i$ de A:

$$\mathbf{x}^{(0)} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

Após t multiplicações por A:

$$A^t\mathbf{x}^{(0)} = \sum_{i=1}^{n} \alpha_i \lambda_i^t \mathbf{v}_i = \lambda_1^t \left(\alpha_1 \mathbf{v}_1 + \sum_{i=2}^{n} \alpha_i \left(\frac{\lambda_i}{\lambda_1}\right)^t \mathbf{v}_i \right)$$

Pelo **Teorema de Perron-Frobenius**, sabemos que *"Para qualquer matriz não negativa irredutível A, o seu maior valor próprio, λ_1, é um número real positivo e os componentes do respetivo vetor próprio, $\mathbf{v}_1$, são todos positivos."*

Como λ_1 é o maior valor próprio, $|\lambda_i/\lambda_1| < 1$ para $i \geq 2$, e portanto $(\lambda_i/\lambda_1)^t \to 0$ quando $t \to \infty$.

Portanto, o vetor $A^t\mathbf{x}^{(0)}$ converge (a menos de um fator de escala) para $\alpha_1\mathbf{v}_1$: **o vetor próprio dominante**.

Algoritmo *Power Iteration*

Em vez de calcular diretamente o vetor próprio, o método *Power Iteration* aproxima-o iterativamente: a partir de vetores iniciais uniformes, cada iteração propaga os scores atuais pelos vizinhos e normaliza o resultado. Portanto, os passos do algoritmo são os seguintes:

1. **Inicialização**: $EC^{(0)}(v) = 1$ para cada um dos vértices do grafo, v.
2. **Propagação**: $EC^{(t+1)}(v) = \sum_{u \in \text{viz}(v)} EC^{(t)}(u)$.
3. **Normalização** (L_∞): Para normalizar os resultados, divida cada um dos novos scores, $EC^{(t+1)}(v)$ pelo valor máximo no vetor $EC^{(t+1)}$, de modo que $\max_v EC^{(t+1)}(v) = 1$.
4. **Convergência**: repita até que $\max_v \left|EC^{(t+1)}(v) - EC^{(t)}(v)\right| < \varepsilon$ ou até que tenha atingido um número máximo de iterações (igual a 100 por defeito). Considere que, por defeito, $\varepsilon = 10^{-6}$.

A convergência é garantida pelo Teorema de Perron-Frobenius. A velocidade de convergência é determinada pelo rácio $|\lambda_2|/\lambda_1$ (entre o segundo maior valor próprio e o primeiro maior valor próprio). Ou seja, quão rápido $(\lambda_2/\lambda_1)^t \to 0$ quando $t \to \infty$. Quanto menor for este rácio, maior é a "folga" entre o primeiro e o segundo valor próprio e menos iterações são necessárias para o algoritmo.

Interpretação: enquanto DC conta apenas os vizinhos diretos, EC pondera esses vizinhos pela sua própria importância. Numa rede social, uma pessoa com poucos contatos mas muito bem ligados pode ter EC elevada e DC baixa. Esta métrica é a base do algoritmo **PageRank**[2] da Google, que estende a EC com uma normalização pelo grau de saída e um fator de amortecimento para lidar com grafos orientados desconexos.

Notas de implementação

- Use duas estruturas de dados (coleções) separadas para os scores de todos os vértices na iteração atual (por ex., numa variável `scores`) e os scores de todos os vértices na iteração seguinte (por ex., numa variável `new_scores`). Não os atualize *in-place* durante o ciclo de propagação.
- A normalização L_∞ consiste em dividir todos os valores de `new_scores` pelo seu máximo (ou seja, o valor máximo da coleção `max(new_scores)`).
- O critério de convergência compara as duas coleções sobre todos os vértices. Use `max(abs(...))`. Repita o algoritmo até que esta diferença máxima seja menor que $\varepsilon = 10^{-6}$ ou até que tenha atingido um número máximo de iterações `max_it` $= 100$.
- Para grafos **desconexos**, aplique a iteração apenas à maior componente conexa e atribua $EC(v) = 0$ aos vértices fora dela.
- O método deve devolver também o número de iterações realizadas, necessário para verificar empiricamente $T_{CC}/T_{EC} \approx n/k$.

5 Betweenness Centrality (BC) – Algoritmo de Brandes

A **Betweenness Centrality** mede com que frequência um vértice v aparece nos caminhos mínimos entre todos os outros pares de vértices:

$$BC(v) = \frac{2}{(n-1)(n-2)} \sum_{\substack{s,t \in V \\ s \neq v \neq t}} \frac{\sigma_{st}(v)}{\sigma_{st}} \tag{7}$$

onde σ_{st} é o número total de caminhos mínimos entre s e t, e $\sigma_{st}(v)$ é o número desses caminhos que passam por v. O fator de normalização $2/[(n-1)(n-2)]$ corresponde ao número de pares não ordenados (s,t) com $s \neq v \neq t$.

O cálculo direto desta fórmula é proibitivo, por isso, o **algoritmo de Brandes (2001)**[4] tenta melhorar a complexidade calculando este valor mais eficientemente em duas fases.

Nomeadamente, para cada nó s de um grafo, $G(V,E)$, temos:

Fase 1 – BFS (modificada): Executa uma BFS a partir de s mantendo, simultaneamente:

- $d[v]$: distância mínima de s a v (inicialmente $d[s] = 0$, $d[v] = -1$ para não visitados). Quando v é descoberto (visitado pela primeira vez) a partir de um vértice predecessor u, fazemos com que: $d[v] = d[u] + 1$.

[4] https://snap.stanford.edu/class/cs224w-readings/brandes01centrality.pdf

- $\sigma[v]$: número de caminhos mínimos de s a v (inicialmente $\sigma[s] = 1$, $\sigma[v] = 0$). Quando v é descoberto pela primeira vez, ou quando é avistado novamente a partir de um vértice predecessor u, e verifica-se que $d[v] = d[u]+1$ (ou seja, encontrou um novo caminho mínimo no grafo a partir de u), fazemos com que: $\sigma[v] \mathrel{+}= \sigma[u]$.
- $pred[v]$: lista de predecessores de v. Adiciona-se u a $pred[v]$ sempre que encontra um novo caminho mínimo (ou seja, avistamos v a partir de u e $d[v] = d[u] + 1$).
- Pilha S: cada vértice v é inserido no topo da pilha por ordem crescente de distância a s. Ou seja, é inserido quando chegar a sua vez de ser processado pela BFS (ou seja, imediatamente após ser removido da fila auxiliar da BFS).

Fase 2 – Retrocesso: Inicialize $\delta[u] = 0$ para todos os vértices u do grafo.

Percorra a pilha S (pelo topo) e, para cada vértice w retirado, itere sobre $v \in pred[w]$:

$$\delta[v] \mathrel{+}= \frac{\sigma[v]}{\sigma[w]} \cdot (1 + \delta[w]) \tag{8}$$

Se $w \neq s$, acumule $\delta[w]$ no contador global da métrica $BC[w]$ (ou seja, $BC[w] \mathrel{+}= \delta[w]$

Após processar todos os nós fonte s do grafo, normalize $BC[v]$ pelo fator $\frac{(n-1)(n-2)}{2}$.

Exemplo: grafo linear A–B–C–D

- $BC(A) = BC(D) = 0$: vértices extremos, nunca intermediários.
- $BC(B)$: todos os caminhos de A para $\{C, D\}$ passam por B: $BC(B) = \frac{2}{3\cdot 2}(1 + 1) = \frac{2}{3}$.
- $BC(C)$: todos os caminhos de $\{A, B\}$ para D passam por C: $BC(C) = \frac{2}{3\cdot 2}(1 + 1) = \frac{2}{3}$.

Interpretação: numa rede social, a BC mede o *poder de intermediação* de um indivíduo. Uma pessoa com BC elevada controla o fluxo de informação entre comunidades da rede, mesmo tendo poucas ligações diretas. É a métrica mais informativa para detetar "pontes estruturais" na rede.

Notas de implementação

- Use, por exemplo, uma lista Python como pilha (`append`/`pop`) (para além da estrutura da fila auxiliar necessária para o algoritmo BFS).
- Os dicionários σ, d, $pred$ e δ devem ser **reinicializados** a cada novo nó fonte s.
- O acumulador global BC é inicializado **uma única vez** antes do ciclo externo e nunca reinicializado.
- Na BFS, ao examinar o vizinho w de u:
  - Se $d[w] < 0$ (não visitado): atribua $d[w] = d[u] + 1$, enfileire w.
  - Se $d[w] = d[u] + 1$ (já descoberto por outro caminho mínimo de igual comprimento): apenas acumule $\sigma[w] \mathrel{+}= \sigma[u]$.
  - Em ambos os casos anteriores (se $d[w] = d[u] + 1$): adicione u a $pred[w]$.
- Para mais detalhes, pode consultar o seu pseudocódigo no artigo onde foi originalmente proposto[4] (página 10). O pseudocódigo está muito claro e completo!

Atenção: use a BFS (não Dijkstra) para calcular distâncias (apesar de funcionar com ambos)! Deste modo, as conclusões sobre esta métrica ficarão coerentes com as conclusões para a métrica CC.

6 Referências

- Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2), 163–177. https://www.tandfonline.com/doi/abs/10.1080/0022250X.2001.9990249
- Wolitzky, A. (2022). *Lectures 1 & 3: Networks and Eigenvector Centrality.* MIT 6.207/14.15. https://ocw.mit.edu
- Alberich, R., Miró-Julià, J., & Rosselló, F. (2002). Marvel Universe looks almost like a real social network. https://arxiv.org/abs/cond-mat/0202174
- Wasserman, S., & Faust, K. (1994). *Social Network Analysis.* Cambridge University Press.
- Closeness Centrality: https://www.sci.unich.it/~francesc/teaching/network/closeness.html
- Eigenvector Centrality: https://pi.math.cornell.edu/~mec/Winter2009/RalucaRemus/Lecture3/lecture3.html